Medi-Saarthi

Team Development & Project Guidelines

SIH 2026 • PS ID SIH26133 • Team NeuralNest

Core principle: Medi-Saarthi is a coordination layer that keeps a rural patient's journey continuous—from first contact to completed care and follow-up. It does not replace doctors or create healthcare capacity.

1. Product Definition

Build: assisted intake, rules-based triage, facility readiness, referral tracking, completion and follow-up. One-line pitch: **Medi-Saarthi is not another healthcare platform; it is the coordination layer that makes the patient's journey continuous.** MVP: Register → Intake → Triage → Readiness → Refer → Accept/Reject → Arrive → Complete → Follow-up.

2. Golden Rules

Build the patient journey first. Do not claim integrations that are not implemented. Use mock adapters for ABDM/FHIR/eSanjeevani/emergency services when access is unavailable. Never present triage as diagnosis. Always show availability verification time and stale-data warnings. Keep the MVP reliable rather than oversized.

3. Core Modules

Frontline Worker: registration, assisted intake, triage, facility search, referrals and follow-up. Pulse-Check: service readiness, timestamp and freshness. Referral Guard: Create → Accept → Arrive → Complete → Close, with rejection reasons. Facility Dashboard: accept/reject, readiness, arrival and completion. Follow-Up and Admin dashboards: pending, missed, high-risk and operational visibility.

4. Four-Member Division

Member 1—Frontend: React/Vite/TypeScript/Tailwind, worker/facility/admin UI. Member 2—Backend: Node/Express/TypeScript, auth, APIs, referral state machine and triage. Member 3—Database & Offline: PostgreSQL, IndexedDB sync queue, seed data and deployment. Member 4—Intelligence & Integration: voice, OCR, readiness, notifications, interoperability mock and testing.

5. Development Order

Sprint 1: foundation, authentication and roles. Sprint 2: patients, facilities and referral workflow. Sprint 3: Referral Guard, Pulse-Check and follow-up. Sprint 4: rules-based triage, voice/OCR and offline sync. Sprint 5: dashboards, testing, demo data, polish and presentation.

6. Technical Standards

Frontend: React + Vite + TypeScript + Tailwind + PWA. Backend: Node.js + Express.js + TypeScript. Database: PostgreSQL. Redis is optional and must not delay the MVP. Voice: Web Speech API prototype. OCR: Tesseract.js prototype with human verification. Maps: Leaflet + OpenStreetMap. Interoperability: FHIR/approved ABDM-compatible design, using adapters/mocks unless real access exists.

7. Referral State Machine

CREATE: source submits. ACCEPT: destination confirms capacity. REJECT: destination records a reason such as specialist/service/slot/equipment unavailable. ARRIVE: patient arrival confirmed. COMPLETE: care completed. CLOSE: follow-up resolved. Every important transition should be timestamped and auditable.

8. Offline-First

PWA stores basic pending operations locally using IndexedDB. A sync queue sends them when connectivity returns. Use unique IDs and timestamps to reduce duplicates/conflicts. Clearly label records as synced or pending.

9. Security & Privacy

Use authentication, role-based authorization, password hashing, input validation, HTTPS, audit logs and data minimization. Use consent-based sharing and approved interoperability workflows. Use only synthetic/demo patient data during the hackathon.

10. UI/UX

Design for frontline workers: large controls, minimal typing, clear status labels and short workflows. Support local-language-friendly interaction. Make referral status prominent. Use explicit states: Available, Unavailable, Pending Verification and Stale. Give feedback after every critical action.

11. Demo Scenario

Create patient → assisted intake → rules-based triage → human verification → Pulse-Check → referral → acceptance → arrival → completion → follow-up. Then show a failure case: required service unavailable → destination rejects → reason recorded → next action/follow-up visible.

12. Judge-Proof Answers

Why ABDM? We complement it by solving operational coordination around the journey. Why eSanjeevani? It supports telemedicine; we focus on readiness, referral acceptance, arrival, completion and follow-up. Can software create beds/doctors? No; we coordinate within available capacity. Percentage claims? Treat them as projected targets until validated. 108/102 integration? Claim only what is actually implemented; otherwise show an integration-ready adapter.

13. Do Not Build for MVP

No autonomous diagnosis, no full hospital-management suite, no production national ABDM integration, no advanced ML before the core workflow works, and no feature that weakens demo reliability.

14. Testing Checklist

Test roles/permissions; invalid registration; every referral state; rejection reasons; audit events; stale availability; offline creation and sync; duplicate sync; mobile/responsive UI; and the complete demo from a clean database.

15. Success Metrics

Referral completion rate, missed referral rate, referral processing time, follow-up completion rate, availability verification rate, frontline data-entry time and percentage of referrals with complete lifecycle events.

16. Final Checklist

■ End-to-end MVP works. ■ Referral Guard visible. ■ Pulse-Check freshness visible. ■ Offline demo works. ■ Voice/OCR has human verification. ■ Synthetic demo data only. ■ Unsupported claims removed. ■ Every member knows their contribution and architecture. ■ Demo is reliable within the pitch time.

Final product definition: Medi-Saarthi coordinates the rural healthcare journey from first contact to completed care and follow-up. The strongest proof is a working closed-loop patient journey where information, referral status, facility readiness and accountability move with the patient across facilities.